

Data Transparency

I assume here that I do not need to spend a lot of time introducing concepts like Data Transparency and open-access data sets. Long story short, increasingly the basic unit of dissemination for academic scholarship and scientific research is not the individual text publication, but a package that can contain data, text, computer code, and multimedia presentations.

“Replication Crisis”

The “Replication Crisis” that became recognized in the 2010s has surely contributed to this development because scientists wanted to evaluate their peers’ research more thoroughly. Instead of publications just providing summaries, scientists want to examine raw data files, data acquisition protocols, data-transformation pipelines, and analytic methods.

Data Transparency Becomes a Priority

Some publishers have started to *require* authors to provide raw data alongside their articles, or else to explicitly state why they are unable to do so (assuming their research involves empirical data sets to begin with). Similar policies have been adopted by funding agencies. For example, the Bill and Melinda Gates foundation requires that all grantees “include a **Data Availability Statement (DAS)** describing where the underlying data can be found ... Underlying data includes all primary data, metadata, and any additional information needed to understand, assess, and replicate the study findings.”

Gates Foundation 2025 Open Access Policy: Data Availability Guide

MIBBI and Other Minimum Information Standards

Data Transparency goes beyond just sharing data once it is acquired. For scientific fields where observations are obtained from physical or laboratory equipment, properly assessing and reproducing research requires relatively detailed understanding of the experimental setup prior to data being captured. Minimum Information standards allow these details to be notated in a structured manner conformant to domain-specific templates. By extension, the overall procedures through which samples and information are transformed — both in the physical and digital/computational realms — should be documented with an eye toward replication.

“Findable, Accessible, Interoperable, Reusable”

According to the popular “FAIR” standard, data should be Findable, Accessible, Interoperable, and Reusable. Arguably the most important consideration here is accessibility. In the typical case, a researcher will learn about some scientific work or experiment from peer-reviewed literature. Once they are reading a specific publication — and should they wish to examine the research in greater detail — the question becomes how readily they can acquire the associated raw data and examine it usefully.

Interoperability means that multiple data sets can be studied and analyzed side-by-side, and Reusability means that data and code can be reused in new experiments — maybe reproducing and/or extending the original data set. But accessibility is a precondition for everything that follows.

Data Sets

Accessibility has multiple dimensions. Starting from a published manuscript, there are usually several steps required to download, deserialize, and study data sets. With FAIR data, these steps can hopefully be accomplished without relying on a lot of external software or manual processing. Ideally, computer code needed to unpack and visualize data contents are included as part of a data set.

FAIR data sets should not rely on users having access to external software, especially software that is not free and open-source. Researchers should be able to assess scientific work without having the same applications or tools that the original scientists employed. For most data sets, raw data needs special deserialization and visualization tools. For FAIR/Accessibility, XML files can't just be viewed as DOM trees in web browsers, and CSV can't just be opened as spreadsheets.

Executable Research Objects

As a standard related to FAIR data, the Research Object specification includes formats such as Research Object Bundles and RO-Crate that govern how data packages are organized and what kinds of metadata they provide. These formats typically include one or more structured files that provide a machine-readable summary of data contents, plus raw data files organized for straightforward access.

More generally, “Research Objects” can refer to any data set published according to FAIR principles, which means that raw data is supplemented with metadata and code to facilitate accessibility, interoperability, and reuse. An “Executable” Research Object prioritizes included code in particular. In particular, an overall data set can be developed as a self-contained computer application with a dedicated entry-point that provides access to different parts and facets of the Research Object. That is, RO code is aggregated into a unified executable package.

Publications Within Research Objects

Individual text publications are still important because they are usually the place where readers learn about research work — e.g., either via a web search or reference in a different publication. But once someone accesses a manuscript that is linked to a Research Object, they may then download that data set and conduct further reading and examination through the Research Object directly. If the original paper is included within that Research Object — in a format such as PDF — readers can view the manuscript via the included file rather than the web document they may have found initially. This creates new possibilities for annotations, cross-references, and interactive content, insofar as words, sentences, and other landmarks in article texts can be linked to data and code elsewhere in the Research Object.

What's “Cross-Reference Logic”?

Note that for a PDF rendering library, each PDF page is just a picture, which can be manipulated and overlaid with generic graphics functionality. The window coordinates of textual elements can be obtained from PDF annotations or by pairing PDF files with metadata compiled using techniques such as `\pdfsavepos`. This means that context menus activated over a PDF page can have different options depending on the contents of words, sentences, or annotations present near or around the mouse-event position. Also, the PDF display can be embedded in GUI windows with additional buttons and controls. Whether via context menus, toolbars, or other controls, Research Object Applications can link PDF windows programmatically with other windows showing videos, digital maps, computer simulations, 3D graphics, and so forth. PDF libraries such as Poppler or XPDF can be included directly in Research Objects, so PDF viewers can be customized for individual data set and shared directly as source-code files.

Text Mining

Alongside the emergence of open-access data sets and Research Objects, there has also been an increasing emphasis on text-mining and AI techniques to expedite academic research. The basic idea is improving the accuracy of search results, by leveraging computationally tractable semantic cues that are more granular than simply searching for specific text strings, or even thematically related text. For example, there are thousands of peer-reviewed articles about Keynesian Economics, and merely identifying some publication as relevant to this overall topic does not clarify whether that document is important to some *particular* researcher's work about this topic. Instead, we'd like to discover more specific classifications, such as resources that *endorse*, *critique*, or neutrally *evaluate* Keynesian Economics; or analyze it from a macroeconomic, or a sociopolitical perspective; or assess it differently from a moral versus quantitative perspective, and so on.

Case Study: “CORD-19”

The CORD-19 corpus, developed by the Allen Institute for Artificial Intelligence (AI2), is a good example of the goals and techniques for text-mining in the context of scientific research. This corpus included over 400,000 freely-available full-text articles available both in human-readable formats and in machine-readable encodings — specifically, S2ORC JSON (S2ORC comes from Semantic Scholar Open Research Corpus). Because Covid research spanned many disparate subject areas — viral morphology, infectivity, genetic profiles and strains, vaccine development, diagnostics, clinical treatments, epidemiology, etc. — automated text-mining tools could, at least in principle, help scientists discover research in other disciplines to help fit the pandemic's pieces together. AI2 did not specifically create those tools, but they curated CORD-19 to be directly accessible from computer software in the hopes that other parties could build text-mining capabilities upon that basis.

Limitations of CORD-19

According to AI2, roughly 30% of CORD-19's full-text documents were shared in structured formats such as JATS-XML, but the majority of publications were only available as PDF files. AI2 therefore had to extract text from those PDFs in order to perform conversion to S2ORC JSON. This is an intrinsically error-prone process due to anomalies within internal PDF text representation, such as hyphenation, ligatures, spurious space characters in some font styles, headers/footers and marginal content that needs to be isolated from the main text, etc. Furthermore, technical content such as chemical formulae or linguistics annotations can be difficult to read properly without working off of structured markup, such as XML. Transcription errors limited the extent to which inter-document connections and conceptual links can be properly recognized. Also, S2ORC only models documents at the paragraph level — not individual sentences — which can limit the application of NLP-related text mining techniques.

AI2 recognized these limitations and even printed a “call to action” encouraging publishers to develop more rigorous text encoding and metadata standards.

But Let's Not Get Too Locked In on Text Mining

I think it is hard to judge the prospects of text mining as a research tool in these ways, sensitive to rhetorical structures and NLP nuance. Nonetheless, the emphasis on machine-readable text encoding — which has been spurred on by text-mining goals — is also important in other contexts.

Even without AI or NLP, rigorous text representation can improve upon native PDF search capabilities; help to implement text-to-data cross-references; and facilitate systematic, actionable semantic annotations schemes within published manuscripts. For instance, intra-document queries can be narrowed in scope to main text; or, conversely, to block quotes, footnotes, section titles, etc. Moreover, special content such as chemical formulae, technical definitions, linguistics example sentences, proper names, GEO-taggable place names/locations, quotations, citations, and etc. can be parsed into structured data objects.

Sentence Objects

There are numerous XML-based and other markup formats for text documents, including JATS, BioC, TEI, S2ORC, and TAGML. Any markup format, however, will encode natural-language text according to generic structures involving trees or graphs, rather than the inherent structure of written text itself. For instance, XML trees are defined in terms of parent and child nodes, siblings, elements' text content, and attribute values. Although there are analogous structures in natural language — for instances, sentences in sequence are similar to sibling XML nodes many textual phenomena can be modeled only partially via XML structure alone.

For these reasons, any markup file should be seen as the *serialization* of textual content, not as the definitive structure of that content. Conceptually, text elements are best understood as typed values in an Object-Oriented context. Moreover, I believe sentence objects — that scale of representation — should be the foundation of such object systems. Sentence objects are also contained inside larger elements (paragraphs, sections, subsections) and can encompass smaller parts (keywords, annotated fragments), but overall text-object systems should pivot around the sentence level.

Sentence Objects and Query Results

This slide shows examples of how text can be modeled in terms of sentence objects. In this specific case, the text for the full-length article I am summarizing with these slides — which is programmed to generate both $\text{\LaTeX}$ and JATS files — also creates serialized versions of sentence objects. The resulting data files can then be parsed — in this case, into C++ objects, objects that can then be used for editing and text analysis. I have found this a useful system when preparing books and articles for publication. On this slide, I show how the objects can be employed for a demo full-text query system. The query is presented within demo code published for this article. This is not a full-fledged query language, but a provisional outline of the technology stack and implementation layers supporting such a language.

Text Elements as Structured Objects

I use the term “Sentence Object Model” to describe text encoding built around sentence objects, but where objects can encompass a spectrum of distinct types both at higher and lower scales. Among sentence objects themselves, there is usually a straightforward structure — where sentences follow one another in sequence, belonging to encompassing paragraphs — but sometimes the structure is more complex. For instance, footnotes may contain multiple sentences or even paragraphs, but are in some sense children of the sentences where their mark appears — or, if it comes after a sentence, the sequence of sentences effectively diverges into main text and footnote content. Similarly, a block quote or enumerated list often functions like a child of the sentence that introduces it, and sometimes the preceding sentence is grammatically incomplete and has the effect of merging with the content that follows it. Also, special content such as quotations, citations, bibliographic entries, and entries/locators for print indexes have their own structural requirements. As these examples suggest, modeling the full organization of textual content requires an ability to represent sometimes complex objects and their interrelationships.

Para Ids and Sentence Boundaries

This slide and the next show a sample document that illustrates one way to use paragraph subdivisions and explicit sentence boundaries. Publishers have started to use paragraph ids instead of page numbers to record index locators during manuscript preparation. The obvious benefit of para-id's is that they remain constant even while pages get renumbered due to changes elsewhere in the document. They are also more granular than page numbers alone, directing us to specific parts of a page — especially in conjunction with 2-column text.

Ordinarily, para-ids are replaced by page numbers in the final publication and are not visible to human readers. However, it is possible that future publishers will choose to keep para-ids as marginal content, to help human readers locate and refer to specific locations on a page. Visible ids introduce hyperlink sources and targets that can lead back and forth between the index and the main text.

Continuation Marks: a, b, c; i, ii, iii; left/right column

With respect to para-ids, one nontrivial question is to define the scope of individual paragraphs. Many paragraphs can include identifiable parts, such as a block quote, followed by further discussion, then perhaps another quotation. This sample document uses a sub-indexing scheme to identify sequential parts of a paragraph within the main text (via letters a, b, c etc.) and block quotes inside paragraphs (via roman *i*, *ii*, etc.). Moreover, special marks identify portions of paragraphs that run onto a new page or the top of a page's right-hand column.

This graphic also shows special marks added to identify sentence boundaries — or, more precisely, the boundaries that have been modeled via metadata. Such marks ordinarily would not be visible to typical readers, but they could be used by authors, editors, or programmers to double-check sentence boundaries as computationally recognized.

Paragraph Subdivisions and Index Locators

Index Locators text

GUI Controls for (Print) Indexing

Index GUI text

GUI Window for Single Index Entry

Single Index Entry Window text

Meta-Index Search

Meta-Indexes text

Reverse-Index Searches

Reverse Indexes text

Viewing Sentence and Annotation Boundaries

Sentences and Annotations text

Data Objects for Executable Research Objects

Data Objects text

Cross-References (again)

More re Cross-References text

GUI Indexing for Research Objects

GUI Indexing text

Computational Support for Text/Data Integration

Text/Data Integration text

Using Scripts to Enhance Research Object Presentations

Scripts text

Language Stacks for Research Objects

Stacks text

Parsing for an Embedded Compiler

Parsing text

Rules and Contexts in Parsing Code

Sentences and Annotations text

The Idea of “Channels”

Channels text

Building Channels Programmatically or From VM Code

Building Channels text

Hidden or “Back Door” Channels

Hidden Channels text

Interface Design Principles for the Research Object Context

Interface Design text

Diamond Open Access

DOA Model text

Translational Science and Translational Medicine

Translational Science text

Science/Humanities Integration

Science/Humanities Integration text